Multi_UniProt_Code_Python3

September 26, 2019

0.1 INTRODUCTION

In Orellana *et al.* 2016, we have used all the crystal structures of a homomeric protein to create an ensemble. What we call an ensemble in this article was a set of structures that are not chimeric, represent the whole protein rather than a single domain of it, and do not contain more than 5 consecutive non-terminal missing residues. In this study we have taken the ensemble definition to one step further and used homologous structures. This tutorial also makes it possible to create an ensemble for heteromeric structures. Meaning that you can provide Uniprot IDs that belong to different proteins, subunits, organism, etc. , given that these sequences are homologous.

In comparison to the Single UniProt Code tutorial, Clustal Omega program must be installed and in addition, a profile alignment with reference sequences generated by the user, outside pdbParser, must be present. The profile alignment ensures that the sequences extracted from the structures are properly aligned. As seen in this example, large deletions or modifications will interfere with the alignment. What we are after is afterall the conservation rate of the wild type sequences.

0.2 HOW TO

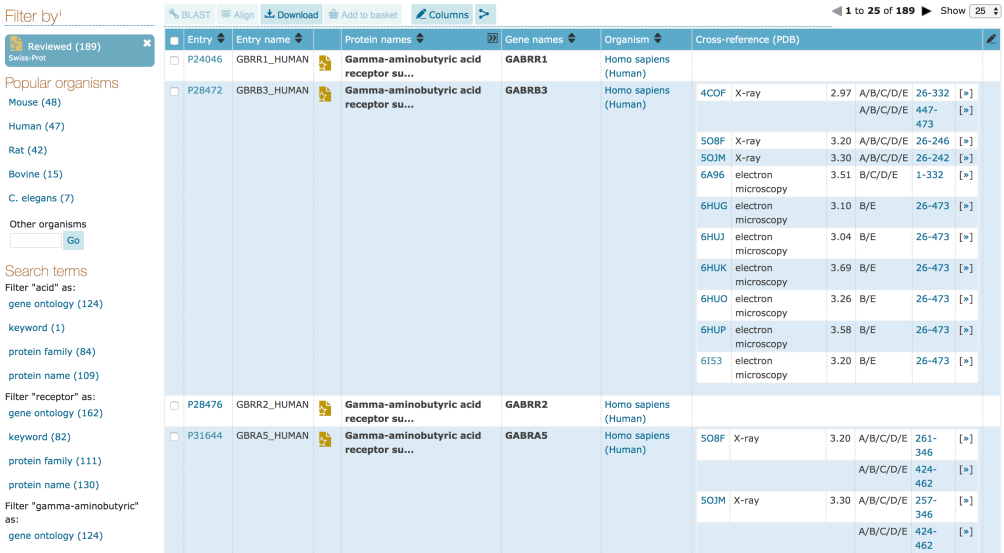
0.2.1 Requirements

Before we get started make sure you have installed the pdbParser package. As mentioned above, you must have Clustal Omega installed. The rest is similar to the Single Uniprot tutorial such that you need to know the UniProt IDs for your systems and the total number of chains that goes into a biological sssembly.

Altough not a requirement, if you want to fix any broken structures you will need a modelling program. We have provided an example script for MODELLER.

0.2.2 Obtaining PDB Files

It can be daunting to find all the available PDB files, and the chains that actually belongs to your protein. The easiset way to do this is to go through the UniProtKB database and note the IDs for pdbParser. If you change the display in UniProtKB, you can already see the entries with the PDB codes as shown below.



title

After finding the UniProt IDs you can run the commands below to gather information on the reference sequence and the available PDBs. This routine was created for heteromeric ensembles mainly but can be used for homomeric ensembles too, in fact, this ensemble has both.

There are quite a lot of files to read and write, a work directory must be created before run time. If you use "." this will use current directory for read/write. Since you could be running this notebook from any location, it is a better practice to provide a full path.

[1]:
```
querylist=["P14867","P47869","P62813","P31644","P28472","P15431","P47870","P18507",
           "P18508","O93430","O75311","G5EBR3","P08219"]
#UniProtKB ID
#P08219 this belongs to bovine, but it is part of 6HUG, in uniprot this was the␣
 ↪id listed for A/D
mer=5 #Total number of chains
cwd="./anionic" #This directory must exist
#exclude=[List of PDB IDs]
#If you already know some structures you want to exclude provide them as a list.
tag="anionic" #Since there are multiple IDs, this tag is used to for writing␣
 ↪files.
```

[2]:
```
from pdbParser import prepENS as pE
info=pE.multiPDBInfo(querylist,mer,tag) #Add the exclude argument to the end.
```

```
CRITICAL:root:Cannot process PDB id 2PR9. It does not contain complete set
```

multiPDBInfo function goes into the UniProt page of the given IDs, and retrives the 3D structures table (as seen in figure below) and the reference sequence to be used for the alignment. For the reasons below it is much easier to prepare the ensemble via a database like UniProtKB: * Because there is a possiblity of all the structures having the same missing residues, the reference sequence is a good control to have. * Within 3D structure table, only the ones with the label "X-ray" are downloaded. Currently we don't support multi-model pdb files. * The relevant chains for this protein is also extracted from this table, this is much easier than reading PDB headers.

After the information is gathered, info object is populated with the possible structures to be used and the reference sequence. You can reach the list from the attributes assigned to this calss. If you want to manipulate this list, add or remove structures, or change the reference sequence you can do so at this stage. Please note that all the functions in prepENS module uses and modifes this class directly. So the attiribute names are important and they should remain unaltered. The values of the attributes , on the other hand, can be manipulated to improve the ensemble.

[3]:
```
print(info.__dict__.keys()) #If you want to see the attributes of the PDBInfo␣
↪class ;)
print(info.querylist) #UniPort IDs used to initialize this class
```

```
dict_keys(['exclude', 'querylist', 'tag', 'mer', 'result', 'refseq', 'broken',
'seqfilename', 'residmapfilename', 'alnfasta', 'coremer', 'coreresids'])
['P14867', 'P47869', 'P62813', 'P31644', 'P28472', 'P15431', 'P47870', 'P18507',
'P18508', 'O93430', 'O75311', 'G5EBR3', 'P08219']
```

The next command downloads the structures from the RCSB database. Depending on the number of structures this function might take a while to finish because after it downloads the files it goes through the steps below for cleaning:

1. It checks for lines with TITLE to find and exclude the chimeric structures.
2. It seperates the biological assemblies within the same PDB file
3. It keeps only the CA coordinates, which are the only relevant atoms for comparison with eBDIMs trajectories. Obtaining only CA coordinates is also a way to clean the PDB files from bound ligands, ions, waters, etc.
4. It extracts the sequence and the residue numbering from the CA coordinates and writes them to text files.

pdbParser relies on multiple sequence alignment to determine the missing residues. This also allows us to ignore the differences in residue numbering between different structures, and non-consecutive numbering (an indication of missing domains, insertions/deletions). The sequence of the structures are extracted from CA coordinates, rather than PDB header. Because a residue might have missing CA coordinate, despite being listed in the sequence records.

The residue numbering information is only used to extract the residues from the the PDB file itself and it is not used in any other way in the alignment.

In the example below you will see that there are more warnings, stating that more structures will be excluded from the ensemble because they are chimeras. Since these structures cannot be used, their information is removed from the info.result dictionary automatically and those files are deleted.

If you check the output folder at this stage, you will find two text files: "_seq.txt" and "_residmap.txt"."_seq.txt" contains the sequences and "_residmap.txt" holds the corresponding residue numbering. These will be used below in the sequence alignment.

[4]:
```
pE.downloadPDB(info,cwd,multiseq=True)
```

```
CRITICAL:root:PDB ID 6CDU is a chimera, skipping this file
CRITICAL:root:PDB ID 6D1S is a chimera, skipping this file
CRITICAL:root:PDB ID 5O8F is a chimera, skipping this file
CRITICAL:root:PDB ID 5OJM is a chimera, skipping this file
```

0.2.3 Multiple Sequence Alignment and Identification of the Shared Region

We need the information within the files created in the previous command for the multiple sequence alignment and for the extraction of a conserved region. Since the reference sequences are highly modified in these structures and they belong to multiple subunits, protein sub families, and organisms I recommend generating a profile alignment via external tools using the reference sequences. You can simply take the portion of the sequences marked with refseq (or write them to a file yourself since they are stored in the info.refseq attribute) for this profile alignment.

Your profile alignment file can have other sequences or have only a few of the reference sequences. What is considered for the core region are only the sequences with refseq tag. However your profile will affect the final alignment gotten from Clustal Omega. The aligned reference sequences are scanned for conserved regions such that if there more than 5 consecutive residues aligned in all than the region is kept and assigned as a block. The final core regions are saved into the info.core_blocks object as a dictionary.

The sequences extracted from the structures are then scanned for gaps only within these blocks and determined as broken or complete. By default an ensemble is skipped if it contains a missing residue within any of these blocks. If you want to see the raw alignment you can do so by printing info.refseqaln or info.structaln . Both of these are pandas data frame objects so you can manipulate them freely. The columns are entirely based on the sequence alignment (with gaps) and they have nothing to do with the actual numbering of the structures or the sequences.

Let's go ahead and generate the initial alignment file.

[5]:
```
clustalopath="/Users/cattibrie_fr/miniconda2/bin/clustalo"
pE.msa(info,cwd,clustalopath,alnf=cwd+"/result.fasta_aln",multiseq=True)
# multiseq=True is required to tell pdbParser that this ensemble has multiple␣
↪reference sequences.
```

```
WARNING:root:6I53_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6HUG_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6HUJ_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6HUK_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6HUO_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6HUP_1.pdb|C| has missing residues in block B1, marked as broken.
WARNING:root:6D6T_1.pdb|E| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|A| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|B| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|C| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|D| has missing residues in block B4, marked as broken.
....
```

As you can see in the first iteration quite a lot of structures were marked as broken/ Visu-ally inspecting the core regions and modifying them to represent the family the best is strongly advised. Since this part requires the experties of the user, it is left as a manual work.
To visualize the alignment you can run info.core_show(). Which will highlight all the core blocks in the alignment and display all the regions. Alternatively you can choose to display a portion of the alignment using positions (again based on the alignment not the sequence number-

ing) or highlt only the selected blocks. To avoid overloading the memory with large alignments, displaying block by block or limiting the positions is highly recommended.

After the core regions are finalized, user can extract the sequences from these structures. As seen here, we can repair some of the structures as they are missing less then five residues. For some other regions we can simply shorten the block (often when the missing region is at the termini). Below are modifications I have made to the conserved regions after inspecting the alignment. I have, for example excluded B3 as it is too short and looks like it is surrounded by a highly variable region.

Some blocks which contained less then half the residues in the structures are automatically removed, for example B7 and B8.

[17]:
```
tes=info.core_show(positions=[info.core_blocks['B1'][0]-5,info.
↪core_blocks['B1'][-1]+5],blocks=['B1'])
tes
```

[ ]:
```
info.core_blocks['B1']=list(range(82,101))
info.core_blocks.pop('B3')
info.core_blocks['B6']=list(range(271,382))
info.core_blocks['B9']=list(range(549,576))
```

[18]:
```
pE.msa(info,cwd,clustalopath,alnf=cwd+"/result.
↪fasta_aln",multiseq=True,updates=True,cores=info.core_blocks)
```

```
WARNING:root:6D6T_1.pdb|E| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|A| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|B| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|C| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|D| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_1.pdb|E| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_2.pdb|P| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_2.pdb|Q| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_2.pdb|R| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_2.pdb|S| has missing residues in block B4, marked as broken.
WARNING:root:4TNV_2.pdb|T| has missing residues in block B4, marked as broken.
WARNING:root:6D6U_1.pdb|E| has missing residues in block B6, marked as broken.
WARNING:root:5CFB_1.pdb|C| has missing residues in block B6, marked as broken.
WARNING:root:6DW0_1.pdb|B| has missing residues in block B9, marked as broken.
WARNING:root:6DW0_1.pdb|E| has missing residues in block B9, marked as broken.
```

To update the broken residue list after tweaking the core region, we can run the pE.msa function again as above, this time setting updates to True and passing along the core regions we want instead of generating them. So now we have a more or less complete set.

Automating ensemble preparation is a tedious task that requires more effort than manual preparation. I have shown and discussed only a few of the problems here. But hopefully pdbParser made your life a bit easier. We can now save the conserved regions and mark the broken structures with getcore, then we are ready for PCA. Remember to set multiseq to True since this ensemble has multiple reference sequences.

[19]:
```
pE.getcore(info,cwd,multiseq=True)
```

0.2.4 Structure Alignment

Since we have extracted a common region, and kept CA coordinates only, all the structures have the same number of atoms. Regardless of the point mutations and the differences in residue numbering we can use almost any alignment tool to superimpose them. For this step you can use your favorite visualizer. In fact it is a good idea to look at the ensemble, and check each structure individually. There could still be unforseen problems. For example in this example 6DW1_1 ensemble has escpaed the checks and was marked as a complete structure despite not having a transmembrane domain. Clearly pdbParser requires un update, which will be hopefully be in place by the time you are following this tutorial.

Other issues could be the subunit placement, for example some GluCl structures have a different subunit placement: anticlockwise vs. clockwise. This makes it difficult to superimpose. To fix this issue you could reorder and rename the chains so that the placement matches the rest of the ensemble.

When you are finished aligning the structures, all you need to do is to combine them, then your ensemble is ready for uploading to the server. Since eBDIMs requires MODEL and END lines in the beginning and the end of the file, you can use a bash loop similar to this to combine them. In fact, to do the structural alignment you can use the ensemble file directly.

[20]:
```
%%bash
touch ./anionic/anionic_ensemble.pdb
for pdb in `ls -1 ./anionic/correct*pdb`
do
echo "MODEL" >> ./anionic/anionic_ensemble.pdb
cat $pdb >> ./anionic/anionic_ensemble.pdb
echo "END" >> ./anionic/anionic_ensemble.pdb
done
```

If you have any questions do not hesitate to contact us. pdbParser, ensemble preperation is a package being developed. Watch out for new features on github

[ ]:
```

```